

Lumify

Bring clarity into your life



Projektleiter:

Christian Riepl

Klasse: **4le**

Schuljahr: **2025/26**

Technische Dokumentation

Inhaltsverzeichnis

Inhaltsverzeichnis.....	1
1 Einleitung	3
1.1 Überblick	3
2 Architektur	4
2.1 Komponenten.....	4
2.2 Architektordiagramm	4
2.3 Zusammenspiel	4
3 Technologie-Stack	5
3.1 Kern-Stack	5
3.2 Eingesetzte Backend-Bibliotheken	5
3.3 Eingesetzte Frontend-Bibliotheken.....	5
4 Backend (.NET-API).....	6
4.1 Projektstruktur	6
4.2 Request-Pipeline	6
4.3 Authentifizierung und Autorisierung	6
4.4 CSRF-Schutz.....	7
4.5 Echtzeit-Kommunikation (SignalR-Hubs)	7
4.6 Weitere Dienste	7
5 Datenbank.....	8
5.1 Konventionen	8
5.2 Entitäten.....	8
5.3 Beziehungen	8
5.4 Datenbank-Indizes.....	9
5.5 View und Trigger	9
6 REST-API	10
6.1 account – Authentifizierung.....	10
6.2 users – Profil und Suche	10
6.3 workspace, notes, todos, events, folders	10
6.4 friendship, chat	10
7 Frontend (Next.js)	11
7.1 Routing-Struktur.....	11
7.2 Context-Provider	11
7.3 Anbindung an die API	11

7.4 Echtzeit im Client.....	11
8 Datenfluss.....	12
8.1 Anmeldung.....	12
8.2 Schreibzugriff mit Echtzeit-Verteilung.....	12
9 Sicherheit	12
10 Betrieb.....	13
10.1 Docker Compose	13
10.2 Konfiguration.....	13
10.3 Ports	13
10.4 Tests	13
11 Glossar.....	13

1 Einleitung

Dieses Dokument beschreibt die technische Umsetzung von Lumify. Darunter den Technologie-Stack, die Architektur und den Aufbau der Datenbank. Es dient als Referenz für Wartung und Weiterentwicklung.

Lumify ist eine Webanwendung zur Organisation von Notizen, ToDo-Listen, Terminen und Dateien. Inhalte können privat oder gemeinsam in geteilten Arbeitsbereichen (Workspaces) verwaltet und über mehrere Clients hinweg in Echtzeit synchronisiert werden.

1.1 Überblick

Schicht	Technologie
Frontend	Next.js 16 / React 19 (TypeScript), eigenes CSS
Backend	ASP.NET Core (.NET 10), C#
Datenbank	MariaDB 11.4 (Entity Framework Core)
Echtzeit	SignalR (WebSocket)
Reverse-Proxy	Caddy 2 (TLS, eine Domain nach außen)
Betrieb	Docker Compose (4 Container)

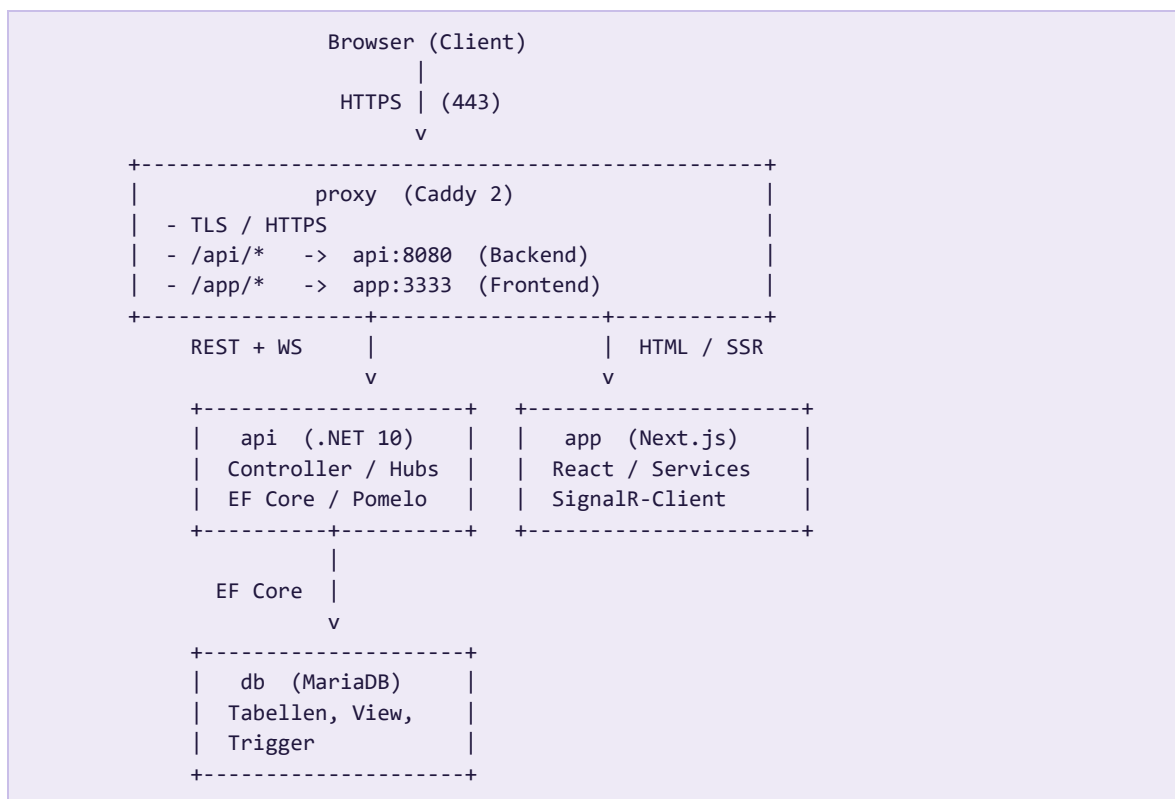
2 Architektur

Lumify ist als Drei-Schichten-Architektur aufgebaut: Frontend, Backend und Datenbank. Jede Schicht läuft im Betrieb in einem eigenen Container. Nach außen ist ausschließlich der Reverse-Proxy erreichbar. Er terminiert TLS und leitet Anfragen intern weiter.

2.1 Komponenten

Container	Technologie	Aufgabe	Port
proxy	Caddy 2	Einziger öffentlicher Endpunkt, TLS, Routing	80 / 443
app	Next.js 16	Frontend, Server-Side Rendering	3333
api	.NET 10	REST-API, SignalR-Hubs, Services, Logik	8080
db	MariaDB 11.4	Persistenz aller Daten	3306

2.2 Architekturdiagramm



2.3 Zusammenspiel

- **Frontend:** rendert die Oberfläche, ruft das Backend über eine zentrale Service-Schicht auf und hält für Echtzeit-Updates SignalR-Verbindungen.
- **Backend:** kapselt die interne Logik, authentifiziert und autorisiert jede Anfrage, persistiert Daten über EF Core und verteilt Änderungen über die Hubs.
- **Datenbank:** speichert alle Daten relational. Eine View und Trigger übernehmen abgeleitete Logik (Fortschritt von ToDo-Listen).

3 Technologie-Stack

Der Kern-Stack umfasst die tragenden Technologien je Schicht. Ergänzende Funktionen werden über Bibliotheken realisiert, die in den Abschnitten 3.2 und 3.3 aufgeführt sind.

3.1 Kern-Stack

Schicht	Technologie
Frontend	Next.js 16, React 19, TypeScript
Styling	Eigenes CSS (CSS Modules + CSS-Variablen) – kein CSS-Framework
Backend	ASP.NET Core (.NET 10), C#
Datenzugriff	Entity Framework Core
Datenbank	MariaDB 11.4
Infrastruktur	Docker Compose, Reverse-Proxy Caddy 2

3.2 Eingesetzte Backend-Bibliotheken

Bibliothek	Version	Zweck
Entity Framework Core	9.0	ORM und Datenbank-Migrationen
Pomelo.EntityFrameworkCore.MySql	9.0	EF-Core-Provider für MariaDB
Microsoft.AspNetCore.SignalR	integriert	Echtzeit-Kommunikation (WebSocket)
JwtBearer / IdentityModel.Tokens.Jwt	10.0 / 8.16	JWT ausstellen und validieren
Otp.NET	1.4	TOTP-Codes (Zwei-Faktor-Authentifizierung)
QRCoder	1.8	QR-Code für die 2FA-Einrichtung
MailKit	4.17	E-Mail-Versand via SMTP

3.3 Eingesetzte Frontend-Bibliotheken

Bibliothek	Zweck
@microsoft/signalr	SignalR-Client für Echtzeit-Updates
@svgr/webpack	Import von SVGs als React-Komponenten
Lokale Schriftarten	Sulphur Point, Borel, Funnel Sans, Vend Sans

Das Aussehen ist vollständig mit eigenem CSS umgesetzt (ein CSS-Modul je Komponente). Ein Framework wie Tailwind wird nicht verwendet.

4 Backend (.NET-API)

4.1 Projektstruktur

Verzeichnis	Inhalt
Controllers	REST-Endpunkte, nach Domänen gruppiert (Account, Users, Workspace, Notes, ...)
Hubs	SignalR-Hubs für die Echtzeit-Kommunikation
Models/EF	Entitätsklassen (Tabellen-Mapping)
Models/DTO	Request- und Response-Klassen sowie Enums
Models/Context	LumifyDbContext: EF-Konfiguration, Beziehungen, Indizes
Services	Fachdienste (Friendship, Presence, SmtEmail)
Logic	Internal Logic (Hashing, JWT, TOTP) und CsrfMiddleware
Migrations	EF-Core-Migrationen inkl. View und Trigger

4.2 Request-Pipeline

Beim Start registriert die Anwendung ihre Dienste (Dependency Injection) und wendet ausstehende EF-Core-Migrationen automatisch an. Jede Anfrage durchläuft anschließend eine festgelegte Middleware-Kette; ihre Reihenfolge ist sicherheitsrelevant:

```
UseStaticFiles    -> /Data/avatars
CsrfMiddleware    -> (Schutz vor gefälschten Anfragen)
UseAuthentication -> (JWT aus Cookie)
UseAuthorization
MapControllers
MapHub            -> /hubs/todos, /hubs/notes, /hubs/events, /hubs/workspaces, /hubs/chat
```

4.3 Authentifizierung und Autorisierung

Nach erfolgreicher Anmeldung stellt das Backend ein JWT aus und legt es in einem HttpOnly-Cookie (session_token) ab. Da das Token im Browser nicht per JavaScript auslesbar ist, ist es gegen XSS-Diebstahl geschützt. Bei jeder Anfrage liest der JwtBearer-Handler das Token aus dem Cookie und prüft Signatur, Issuer, Audience und Gültigkeit.

- **Passwörter** werden mit HMAC-SHA512 und individuellem Salt verschlüsselt gespeichert. Das Klartext-Passwort wird also nie festgehalten.
- **Gültigkeit:** das Token gilt standardmäßig 480 Minuten. (8h)
- **Rollen:** zwei Rollen (User, Admin); geschützte Endpunkte sind mit [Authorize] markiert.

4.3.1 Zwei-Faktor-Authentifizierung (TOTP)

Optional lässt sich pro Konto eine TOTP-basierte Zwei-Faktor-Authentifizierung aktivieren. Bei der Einrichtung wird ein Secret als QR-Code bereitgestellt. Erst nach Bestätigung eines gültigen Codes wird 2FA aktiv. Danach liefert der erste Anmeldeschritt nur einen kurzlebigen MFA-Token, den der zweite Schritt zusammen mit dem sechsstelligen Code gegen eine Sitzung eintauscht.

4.3.2 Passwort-Reset

Ein angefordertes Reset-Token wird nur als SHA-256-Hash gespeichert; der Klartext geht ausschließlich per E-Mail an den Nutzer. Tokens sind einmalig verwendbar und laufen ab.

4.4 CSRF-Schutz

Da die Sitzung in einem Cookie liegt, ergänzt eine eigene Middleware einen Schutz nach dem Double-Submit-Token-Verfahren: Bei schreibenden Anfragen (POST, PUT, PATCH, DELETE) muss der Wert des Cookies XSRF-TOKEN zusätzlich im Header X-CSRF-Token gesendet werden. Stimmen beide nicht überein, wird die Anfrage abgewiesen. Lesende Anfragen und die Anmelde-Endpunkte sind ausgenommen.

4.5 Echtzeit-Kommunikation (SignalR-Hubs)

Fünf Hubs verteilen Änderungen an alle Clients mit demselben Kontext. Ein Client tritt der Gruppe seines aktiven Workspace bei. Ändert ein Mitglied einen Inhalt, sendet das Backend das Ereignis an alle Mitglieder dieser Gruppe. Im privaten Bereich wird keine Verbindung aufgebaut.

Hub	Pfad	Zuständigkeit
ToDoHub	/hubs/todos	Listen und Einträge
NoteHub	/hubs/notes	Notizen, Textblöcke, Links
EventHub	/hubs/events	Kalendertermine
WorkspaceHub	/hubs/workspaces	Mitglieder, Workspace-Änderungen, Präsenz
ChatHub	/hubs/chat	Chat-Nachrichten je Raum

4.6 Weitere Dienste

- **SmtpEmailService:** E-Mail-Versand über einen konfigurierbaren SMTP-Server (MailKit).
- **PresenceService:** verwaltet als Singleton den Online-Status anhand der Hub-Verbindungen.
- **Avatare:** werden hochgeladen, unter Data/avatars gespeichert und als statische Dateien ausgeliefert.

5 Datenbank

Die Datenhaltung erfolgt in MariaDB über Entity Framework Core. Das Schema wird vollständig durch EF-Core-Migrationen definiert und beim Start der API automatisch angewendet.

5.1 Konventionen

- **Primärschlüssel** sind anwendungsseitig vergebene String-IDs. (GUID)
- **Besitz und Sichtbarkeit:** Inhalte tragen eine OwnerID und eine optionale WorkspaceID. Ist die WorkspaceID leer, ist der Inhalt privat. Sonst gehört er dem Workspace.
- **Zeitstempel:** CreatedAt und UpdatedAt werden durchgängig geführt.
- **Soft-Delete:** Datensätze werden über DeletedAt als gelöscht markiert statt entfernt. Abfragen filtern auf DeletedAt IS NULL. (Nachvollziehbarkeit)

5.2 Entitäten

Tabelle	Beschreibung	Wesentliche Felder
User	Benutzerkonto	Username*, Email*, PasswordHash, Role, AvatarUrl, TotpEnabled
Friendship	Freundschaft (paarweise)	UserLowID, UserHighID, RequesterID, AddresseeID, Status
Workspace	Geteilter Arbeitsbereich	OwnerID, Name
WorkspaceMember	Mitgliedschaft (n:m)	WorkspaceID, UserID, Role
Folder	Ordner (verschachtelbar)	OwnerID, WorkspaceID?, ParentFolderID?, Name
Note	Notiz	OwnerID, WorkspaceID?, FolderID?, Name
Note_TextBlock	Textblock einer Notiz	NoteID, Type, Content, CodeLanguage, NotePos
Note_LinkItem	Link einer Notiz	NoteID, Label, Url, NotePos
ToDoList	ToDo-Liste	OwnerID, WorkspaceID?, Name, Status, IsArchived
ToDoEntry	Eintrag einer Liste	ToDoListID, OwnerID, Name, Status
Event	Kalendertermin	OwnerID, WorkspaceID?, Name, StartDate, EndDate, IsAllDay
ChatMessage	Chat-Nachricht	RoomID, SenderID, Content, CreatedAt
PasswordResetToken	Reset-Token (gehasht)	UserID, TokenHash*, ExpiresAt, UsedAt

5.3 Beziehungen

Den Kern bildet die Entität User. Die wichtigsten Beziehungen im Überblick:

User	1---*	Note, ToDoList, Event, Folder	(OwnerID)
Workspace	1---*	Note, ToDoList, Event, Folder	(optional, WorkspaceID)
User	*---*	Workspace über WorkspaceMember	(n:m-Mitgliedschaft)
Folder	1---*	Folder	(ParentFolderID, Baum)
Folder	1---*	Note	
Note	1---*	Note_TextBlock, Note_LinkItem	
ToDoList	1---*	ToDoEntry	
User	*---*	User über Friendship	(UserLow / UserHigh)
User	1---*	ChatMessage	(SenderID)

Die beiden Nutzer-IDs werden immer in derselben Reihenfolge als (UserLowID, UserHighID) abgelegt, sodass jedes Paar nur einmal vorkommen kann. Requester und Addressee halten Richtung und Status fest.

5.4 Datenbank-Indizes

Indizes beschleunigen Abfragen. Mehrere davon sind eindeutig (unique) und verhindern doppelte Daten. Die wichtigsten Indizes:

Constraint	Tabelle	Bedeutung
ux_user_email, ux_user_username	User	E-Mail und Username eindeutig
ux_workspacemember_unique	WorkspaceMember	Ein User je Workspace nur einmal Mitglied
ux_friendship_pair	Friendship	Ein Freundschaftspaar nur einmal
ux_note_... / ux_folder_...	Note, Folder	Name je Ordner bzw. Workspace eindeutig
ux_passwordresettoken_hash	PasswordResetToken	Token-Hash eindeutig auffindbar

5.5 View und Trigger

Der Fortschritt einer ToDo-Liste wird datenbankseitig gepflegt. Die View vw_todolist_progress aggregiert je Liste Anzahl, erledigte und offene Einträge sowie den Fortschritt in Prozent. Drei Trigger auf TodoEntry (INSERT, UPDATE, DELETE) halten TodoList.Status konsistent: Sind alle aktiven Einträge erledigt, gilt die Liste als erledigt (Status 2), sonst als offen (Status 1).

6 REST-API

Die Endpunkte folgen dem Muster `/controller/action` und liefern JSON. Bis auf die Anmelde-Endpunkte erfordern alle Aufrufe eine gültige Sitzung.

6.1 account – Authentifizierung

Methode	Endpunkt	Beschreibung
POST	/account/loginUser	Anmeldung (Schritt 1)
POST	/account/verifyTotpLogin	Anmeldung Schritt 2 (TOTP)
POST	/account/registerUser	Registrierung
POST	/account/logoutUser	Abmeldung
POST	/account/requestPasswordReset · resetPassword	Passwort zurücksetzen
POST	/account/setupTotp · confirmTotp · disableTotp	2FA verwalten
GET	/account/whoAmI · mfaStatus	Sitzungs-/2FA-Status

6.2 users – Profil und Suche

Methode	Endpunkt	Beschreibung
GET	/users/getUserProfile · getUserAccountInfo	Eigenes Profil/Konto
PATCH	/users/saveUserProfile	Profil aktualisieren
POST	/users/saveUserAvatar	Avatar hochladen
DELETE	/users/deleteAccount	Konto löschen
GET	/users/searchUsers · searchAvailableUsersForWorkspace	Nutzersuche
GET	/users/getLast5Modified... (Todos/Events/Notes)	Dashboard-Daten

6.3 workspace, notes, todos, events, folders

Methode	Endpunkt	Beschreibung
POST/PATCH/DELETE	/workspace/...	Workspaces und Mitglieder verwalten
POST/PATCH/DELETE	/notes/...	Notizen, Textblöcke und Links verwalten
POST/PATCH/DELETE	/todos/...	Listen und Einträge verwalten
POST/PATCH/DELETE	/events/... · /folders/...	Termine und Ordner verwalten
GET	.../getAllof User Workspace	Inhalte privat oder je Workspace laden

6.4 friendship, chat

Methode	Endpunkt	Beschreibung
POST/PATCH	/friendship/...	Anfragen senden, annehmen, ablehnen, entfernen
GET	/friendship/getFriendsOfUser	Freunde und Anfragen abrufen
GET	/chat/getMessagesOfRoom	Nachrichtenverlauf eines Raums

7 Frontend (Next.js)

7.1 Routing-Struktur

Das Frontend nutzt den App Router. Route Groups gruppieren Layouts, ohne die URL zu verändern: (auth) für den Anmeldebereich und (app) für den authentifizierten Bereich; darin kapselt (space) die Arbeitsbereiche, die für privat und Workspace identisch funktionieren.

Route	Inhalt
(auth)/Auth	Login, Registrierung, 2FA, Passwort-Reset
(app)/Dashboard	Workspaces, Freunde, zuletzt geänderte Inhalte
(app)/(space)/SpaceHub	Einstieg in einen Space
(app)/(space)/Notes · Todos · Events	Notizen, Todos, Kalender

7.2 Context-Provider

Übergreifender Zustand wird über React-Context-Provider bereitgestellt. Sie lassen sich nach Aufgabe gruppieren:

- **Anwendungszustand:** SpaceProvider (aktiver Workspace), PresenceBridge (Online-Status), ThemeProvider (Farbschema).
- **Oberfläche:** ToastProvider, TooltipProvider, AlertProvider sowie ModalProvider und AccountModalProvider für Dialoge und Overlays.
- **Medien:** MusicProvider für optionale Hintergrundmusik.

7.3 Anbindung an die API

Sämtliche API-Aufrufe sind in einer Service-Schicht (services/api) gebündelt und nutzen den zentralen Wrapper saveFetch. Dort sind die übergreifenden Aspekte einmalig geregelt:

- Das Session-Cookie wird über credentials: "include" stets mitgesendet.
- Bei schreibenden Anfragen wird der CSRF-Token als Header X-CSRF-Token gesetzt.
- Bei abgelaufener Sitzung (HTTP 401) erfolgt automatisch Abmeldung und Weiterleitung zur Anmeldeseite – außer bei den Anmelde-Aufrufen selbst.

7.4 Echtzeit im Client

Pro Hub existiert ein React-Hook (zB. useTodoHub), der die SignalR-Verbindung aufbaut (withCredentials, automatischer Reconnect), der passenden Gruppe beitrifft und auf eingehende Ereignisse reagiert. Diese aktualisieren den lokalen Zustand und damit die Ansicht. Beim Verlassen der Seite wird die Verbindung geschlossen.

8 Datenfluss

8.1 Anmeldung

```
Frontend (LoginPanel)
-> POST /api/account/loginUser      (credentials: include)
-> Caddy -> api
-> AccountController.LoginUser
  - Passwort gegen HMAC-SHA512-Hash prüfen
  - 2FA aktiv? ja -> MFA-Token (Schritt 2 nötig)
                nein -> JWT ausstellen
  - Set-Cookie: session_token (HttpOnly) + XSRF-TOKEN
<- 200 OK -> Weiterleitung /Dashboard
```

8.2 Schreibzugriff mit Echtzeit-Verteilung

```
Client A (ToDo abhaken)
-> PATCH /api/todos/saveTodoEntry
Header: X-CSRF-Token Cookie: session_token
-> CsrfMiddleware ok -> Authentifizierung ok
-> TodoListsController.SaveTodoEntry
  - EF Core: Eintrag-Status aktualisieren
  - DB-Trigger setzt TodoList.Status neu
  - Hub: Ereignis an Gruppe workspace:{id}
<- 200 OK

Client B (gleicher Workspace)
<- SignalR 'TodoEntryUpdated' -> Ansicht aktualisiert sich
```

9 Sicherheit

Schutzziel	Maßnahme
Sitzungstoken	JWT im HttpOnly-Cookie; im Betrieb zusätzlich Secure-Flag
XSS-Token-Diebstahl	Token für JavaScript nicht zugänglich
CSRF	Double-Submit-Token (Cookie XSRF-TOKEN + Header X-CSRF-Token)
Passwortschutz	HMAC-SHA512 mit individuellem Salt, keine Klartextspeicherung
Kontoübernahme	Optionale Zwei-Faktor-Authentifizierung (TOTP)
Passwort-Reset	Token nur als SHA-256-Hash gespeichert, einmalig und ablaufend
Transport	TLS über Caddy, Weiterleitung von HTTP auf HTTPS
Zugriffskontrolle	[Authorize] auf geschützten Endpunkten, Rollen User/Admin
Geheimnisse	DB-Passwörter und JWT-Secret in .env, nicht im Repository

10 Betrieb

10.1 Docker Compose

Der gesamte Stack wird über Docker Compose betrieben. Vier Container (proxy, app, api, db) teilen sich ein internes Netzwerk. Die Startreihenfolge ist definiert: api nach erfolgreichem Healthcheck der db, app nach api, proxy zuletzt. Persistente Daten liegen in Volumes (db_data, api_data, caddy_data, caddy_config).

10.2 Konfiguration

Geheime Werte (DB-Passwörter, JWT-Secret) stehen in einer .env-Datei (Vorlage: .env.example) und werden nicht versioniert. Weitere Einstellungen (Connection String, JWT-Parameter, Frontend-URL) erfolgen über Umgebungsvariablen im Compose-File.

10.3 Ports

Umgebung	Frontend	Backend	Zugang
Entwicklung	5333	5331	direkt während der Entwicklung
Betrieb (Docker)	3333	8080	Port 80/443 über Caddy

10.4 Tests

Für die Backend-Logik existiert ein xUnit-Testprojekt (lumify.tests).

11 Glossar

Begriff	Bedeutung
Workspace	Geteilter Arbeitsbereich; Inhalte gehören allen Mitgliedern
Privater Bereich	Persönlicher Bereich eines Nutzers (WorkspaceID leer)
Reverse-Proxy	Vorgeschalteter Verteiler (Caddy), einziger öffentlicher Endpunkt
JWT	JSON Web Token. Signiertes Sitzungstoken
HttpOnly-Cookie	Cookie, das für JavaScript nicht zugänglich ist
CSRF	Cross-Site Request Forgery. Per Double-Submit-Token abgewehrt.
SignalR / Hub	Technik bzw. Endpunkt für die Echtzeit-Kommunikation (WebSocket)
Context-Provider	React-Mechanismus zur Bereitstellung übergreifenden Zustands
ORM	Object-Relational Mapping. Hier Entity Framework Core
Soft-Delete	Logisches Löschen über das Feld DeletedAt
TOTP	Time-based One-Time Password; Verfahren der Zwei-Faktor-Authentifizierung